

# Welcome & Module Induction

**What we learned previously:** In Module 13, we mastered Asynchronous Programming, allowing our code to handle thousands of connections without blocking.

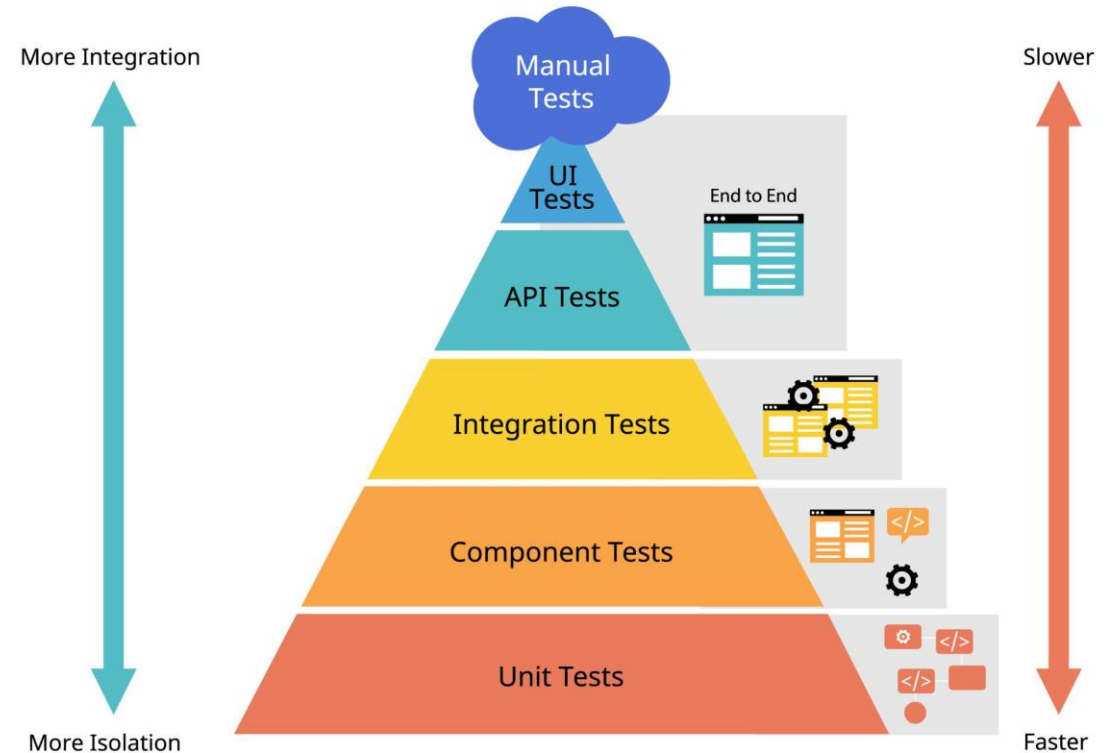
**What we are learning today:** We are learning how to prove our code works. We will explore **Unit Testing** (the foundation), **Pytest** (the industry standard), and **Mocking** (simulating the world).

**Why this will be helpful:** In a professional lab, you don't "hope" your code works; you "verify" it. If you update a function in your AI pipeline, how do you know you didn't accidentally break something else? Automated tests allow you to hit one button and check 1,000 logic points in seconds.

# Unit Testing & the unittest Library

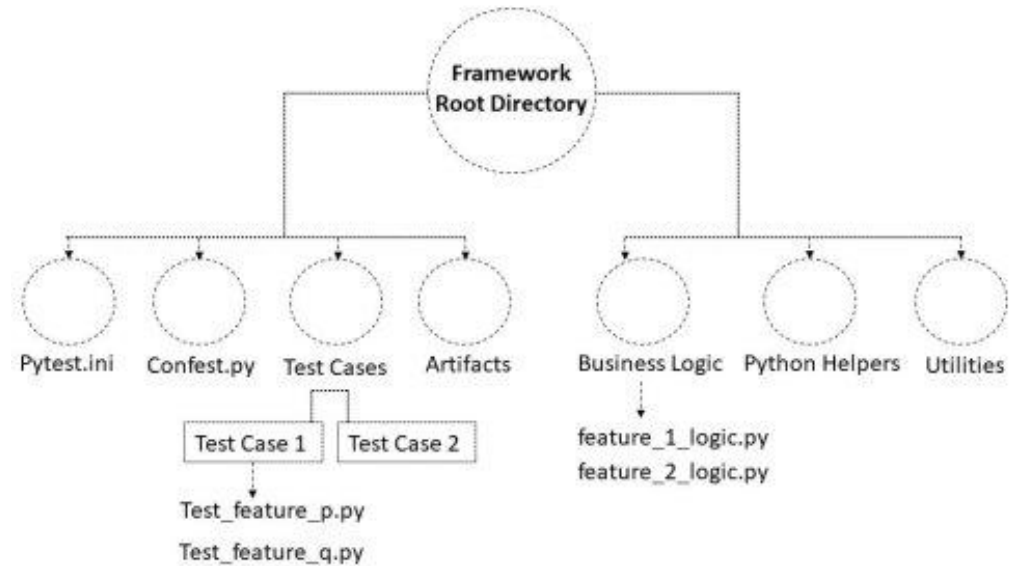
---

- **The Concept:** A "Unit" is the smallest piece of testable code (usually a single function).
- **The Foundation:** unittest is built directly into Python. It uses a "Class-based" approach where you define what a "Success" looks like using **Assertions** (e.g., `assertEqual`, `assertTrue`).
- **The Cycle:** Test => Fail => Write Code => Pass.



# Professional Testing with Pytest

- **Why Pytest?** While unittest is good, pytest is faster, more powerful, and easier to read. It allows you to use simple assert statements instead of complex method calls.
- **The Industry Standard:** 90% of professional Python projects use Pytest because it can automatically find and run all your tests with a single terminal command.



# Mocking & Patching (The "Flight Simulator")

---

- **The Problem:** How do you test a function that sends an email, deletes a database, or calls an expensive OpenAI API? You don't want to actually send 100 emails every time you run a test.
- **The Solution:** Mocking. We create a "Fake" object that mimics the behavior of the real one.
- **The Analogy:** It's like a flight simulator. The pilot can practice landing in a storm without actually risking a million-dollar airplane.

